

Шпаргалка по DevSecOps

Что это такое и чем отличается от «безопасности в конце», какие проверки ставить на каждом этапе, какие инструменты бывают, как не утопить команду в ложных срабатываниях и по каким метрикам понимать, что подход работает.

Суть подхода

- Безопасность встроена в обычный процесс разработки, а не пристроена перед релизом.
- Чем раньше найден дефект, тем дешевле исправление: правка в ветке против инцидента в рабочей среде.
- Ответственность общая: команда разработки отвечает за безопасность своего кода, специалисты дают инструменты и экспертизу.
- Проверки автоматические и быстрые — то, что требует ручного запуска, перестают делать.
- Цель не «ноль находок», а управляемый и понятный риск.

Было и стало

Раньше	Аудит за неделю до релиза, длинный отчёт, сдвиг сроков, спорные приоритеты
Теперь	Проверки на каждом коммите и запросе на слияние, находки приходят автору сразу, в понятном ему виде
Что остаётся ручным	Моделирование угроз, разбор архитектуры, пентест перед крупным запуском — автоматика их не заменяет

Проверки по этапам

Проектирование	Моделирование угроз: что защищаем, кто атакует, где границы доверия
Написание кода	Подсказки в редакторе, поиск секретов до коммита, безопасные шаблоны и библиотеки
Запрос на слияние	Статический анализ, проверка зависимостей, обзор изменений в чувствительных местах
Сборка	Сканирование образов, формирование перечня компонентов, подпись артефактов
Тестовый контур	Динамическая проверка приложения, проверка конфигурации инфраструктуры
Выкладка	Проверка подписи, ограниченные права конвейера, подтверждение для рабочей среды
Эксплуатация	Наблюдение, обнаружение аномалий, управление уязвимостями, реагирование

Виды анализа

SAST	Статический анализ исходного кода: находит внедрение запросов, небезопасные вызовы, слабую криптографию
SCA	Анализ состава: уязвимые и несовместимые по лицензии зависимости
DAST	Проверка работающего приложения снаружи, без доступа к коду
IAST	Анализ изнутри во время тестов: меньше ложных срабатываний
IaC-сканирование	Ошибки в Terraform, Kubernetes и облачной конфигурации до применения
Сканирование образов	Уязвимые пакеты и небезопасные настройки в контейнерах
Поиск секретов	Ключи и пароли в коде и истории репозитория

Инструменты

- **Код:** Semgrep, CodeQL, Bandit (Python), gosec (Go), ESLint с правилами безопасности.
- **Зависимости:** Dependabot, Renovate, OWASP Dependency-Check, Trivy.
- **Образы и контейнеры:** Trivy, Gype, Docker Scout, Syft для перечня компонентов.
- **Инфраструктура:** tfsec, Checkov, kube-bench, Conftest.
- **Секреты:** gitleaks, TruffleHog, серверные проверки при push.
- **Работающее приложение:** OWASP ZAP, Nuclei.
- **Подпись и происхождение:** cosign, SLSA-подход, аттестации сборки.

Секреты и доступы в конвейере

- Хранилище секретов вместо переменных в настройках проекта, где это возможно.
- Короткоживущие маркеры и OIDC-доступ к облаку вместо постоянных ключей.
- Сборка не имеет прав на рабочую среду; выкладка — отдельная задача с отдельными правами.
- Для запросов из форков секреты недоступны — это защита, а не поломка.
- Журналы маскируют значения, но лишний echo легко всё раскрывает.

Чтобы это не превратилось в шум

- Блокируйте сборку только на критичном и подтверждённом; остальное — в отчёт и в очередь задач.
- Настраивайте правила под проект: каждое ложное срабатывание снижает доверие ко всей системе.
- Задайте порог: например, блокируем уязвимости высокого уровня с доступным исправлением.
- Находки должны приходить туда, где работает автор: в запрос на слияние, а не в отдельный портал.
- Заведите правило исключений с обоснованием и сроком пересмотра, иначе их будут молча обходить.
- Держите список исключений маленьким и видимым.

Моделирование угроз за час

1. Что мы строим
Простая схема: компоненты, потоки данных, внешние зависимости
2. Где границы доверия
Каждая граница — место для проверки прав и данных
3. Что может пойти не так
Подмена, подделка, раскрытие, отказ, повышение прав — по каждому потоку
4. Что делаем
Меры, принятые риски и то, что откладываем осознанно
5. Записать и вернуться
Короткий документ рядом с кодом; пересмотр при крупных изменениях

Метрики

- **Время до исправления** по уровням критичности — главный показатель зрелости.
- **Доля исправленного** от найденного за период.
- **Возраст открытых находок** — не растёт ли хвост.
- **Доля ложных срабатываний** — показывает, стоит ли доверять инструменту.
- **Охват**: какая часть репозитория и образов реально проверяется.
- **Время на конвейер**: проверки не должны заметно замедлять выкладку.

С чего начать

1. Поиск секретов в репозиториях и в истории — самая быстрая отдача.
2. Автоматические обновления зависимостей.
3. Сканирование образов в конвейере с отчётом, пока без блокировки.
4. Статический анализ на запросах на слияние, с правилами под проект.
5. Моделирование угроз для одного ключевого сервиса — как пример для остальных.
6. Порядок реагирования на инцидент, записанный и опробованный.